OpenClaw-RL: Train Any Agent Simply by Talking

Yinjie Wang[*], **Xuyang Chen**[*], **Xiaolong Jin**[*], **Mengdi Wang**[†], **Ling Yang**[†]

https://github.com/Gen-Verse/OpenClaw-RL

Every agent interaction generates a **next-state signal**, namely the user reply, tool output, terminal or GUI state change that follows each action, yet no existing agentic RL system recovers it as a live, online learning source. We present **OpenClaw-RL**, a framework that employs next-state signals to optimize personal agents online through infrastructure and methodology innovations. On the infrastructure side, we extend existing RL systems to a server–client architecture where the RL server hosts the policy behind an inference API and user terminals stream interaction data back over HTTP. From each observed next state, the system extracts two complementary training signals, evaluative and directive, via a separate asynchronous server so that neither signal extraction nor optimization blocks inference. On the methodology side, we introduce a **hybrid RL** objective that unifies both signal types in a single update: directive signals provide richer, token-level supervision but are sparser, while evaluative signals are more broadly available. To stabilize distillation under teacher–student mismatch, we propose **overlap-guided hint selection**, which picks the hint whose induced teacher distribution maximally overlaps with the student's top-k tokens, together with a log-probability-difference clip that bounds per-token advantages. Applied to **personal agents**, OpenClaw-RL enables an agent to improve simply by being used, recovering conversational signals from user re-queries, corrections, and explicit feedback. Applied to **general agents**, OpenClaw-RL is the first RL framework to unify real-world agent settings spanning terminal, GUI, SWE, and tool-call environments, where we additionally demonstrate the utility of next-state signals in long-horizon settings.

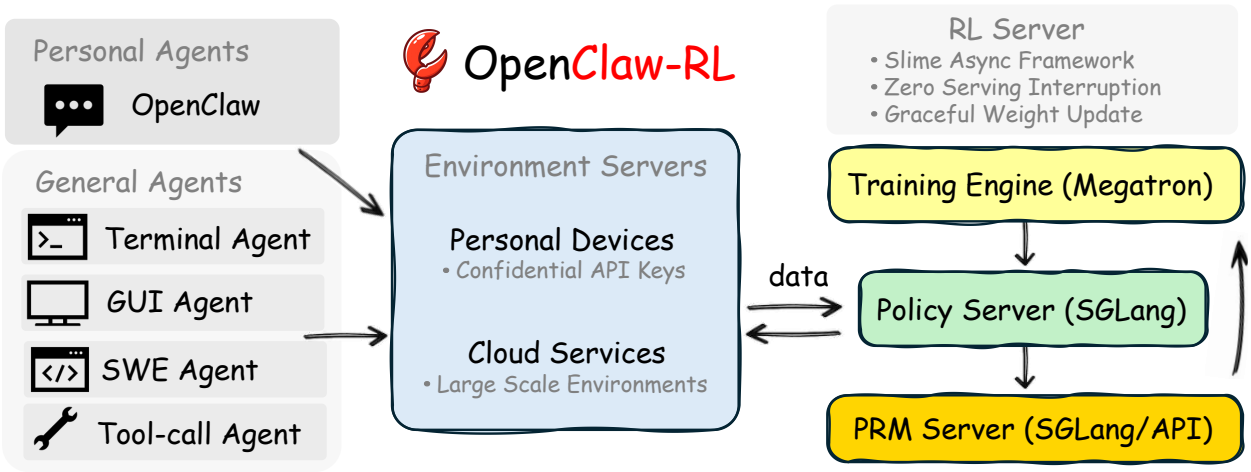


Figure 1 | **OpenClaw-RL infrastructure overview.** Interaction streams come from two agent types: Personal Agents (conversational, personalized), hosted on personal devices, and General Agents (terminal, GUI, SWE, and tool-call agents), hosted on cloud services. The collected samples flow into our RL server built on the asynchronous **slime** framework, which consists of four *decoupled components*: (1) the environment server, (2) **PRM / Judge** for reward computation, (3) **Megatron** for policy training, and (4) **SGLang** for policy serving. The environment for personal agents is simply the users' personal devices, which connect to the RL server over HTTP with confidential API keys. The environments for general agents are hosted on cloud services to enable scalable parallelization.

[*]Equal contribution; [†]Corresponding authors; *Main contact: yangling0818@163.com*

Contents

1 Introduction 3

2 OpenClaw-RL Infrastructure: Unified System for Personal and General Agents 5
- 2.1 Flexible Server–Client Architecture for Live Agents . . . 5
- 2.2 Asynchronous Signal Extraction from Next States . . . 5
- 2.3 Scalability: From Personal Agents to Real-World Agent Deployment . . . 6

3 Methodology: Learning from Next-State Signals 6
- 3.1 Two Complementary Signals and A Hybrid RL Objective . . . 7
- 3.2 Overlap-Guided Hint Selection . . . 8
- 3.3 Step-wise Reward for General Agentic RL . . . 9
  - 3.3.1 Why Process Rewards Are Vital for Agentic Tasks . . . 9
  - 3.3.2 Integrate Outcome and Process Rewards . . . 9

4 Experiments 10
- 4.1 Personal Agent Setup . . . 10
- 4.2 General Agent Setup . . . 10
- 4.3 Hybrid RL Extension Setup . . . 11
- 4.4 OpenClaw-RL Effectively Optimizes Personal Agents . . . 11
- 4.5 Unified RL Framework Across Terminal, GUI, SWE, and Tool-Call . . . 12
- 4.6 Hybrid RL is More Efficient . . . 13
- 4.7 Hybrid RL Generalizes to Agentic RL . . . 13
- 4.8 Overlap-Guided Hint Selection Improves Efficiency and Stability . . . 13
- 4.9 Log-Probability Difference Is Vital for Stability . . . 14
- 4.10 Ablation on k and Support Set S_i . . . 14
- 4.11 Explore Different Policy and Reward Models . . . 14

5 Related Work 15
- 5.1 RL for LLMs . . . 15
- 5.2 Agentic RL and Tool-Use . . . 15
- 5.3 Process Reward Models . . . 15
- 5.4 On-Policy Distillation and Hindsight Methods . . . 15
- 5.5 RL Training Infrastructure . . . 16

6 Conclusion 16

A Experiment Details 21
- A.1 Personal RL Configurations . . . 21
- A.2 General Agentic RL Hyperparameters . . . 21
- A.3 Hybrid RL Extension Experiment Configurations . . . 22
- A.4 Token-level OPD . . . 22
- A.5 Token-Level Log-Probability Shift Analysis . . . 22
- A.6 Algorithm Pseudocode . . . 23
- A.7 Personal RL Prompt Templates . . . 24

B Additional Experiment Results 31
- B.1 More Optimization Examples . . . 31
- B.2 Response Length and Truncation Ratio Example . . . 32
- B.3 PRM Ablation Results . . . 33

C OPD Objective is Token-level KL 33

1. Introduction

Agentic systems powered by large language models (LLMs) have been shown to significantly improve productivity and are now widely deployed in industrial settings (Anthropic, 2025; OpenAI, 2025; OpenClaw, 2026). The data generated through agent usage, such as user replies, tool execution results, and GUI state transitions, constitutes a valuable source of insights for improving agent frameworks (Lee et al., 2026; Zhang et al., 2024), building memory systems (Chhikara et al., 2025; Packer et al., 2023; Rasmussen et al., 2025; Wang and Chen, 2025; Xu et al., 2025), and producing high-quality training data for the models (Ouyang et al., 2022). However, both the infrastructure and methodology for leveraging such usage data to improve language models in real time remain little explored.

The effectiveness of real-time learning for agentic models hinges on the careful design of both infrastructure and methodology. On the infrastructure side, the RL server must be flexible enough to integrate with users' diverse and evolving agentic frameworks, and the optimization process must run asynchronously so as not to block inference-time usage. On the methodology side, the algorithm must fully exploit the extracted training signals while maintaining stability throughout optimization. In this work, we propose **OpenClaw-RL**, a framework that combines infrastructure and algorithmic innovations to achieve efficient and stable online optimization for personal agents.

We extend existing RL infrastructure (Fu et al., 2025; Hu et al., 2024; Sheng et al., 2025; Zhu et al., 2025), which typically assumes batch data collection on the RL server, to support continuous learning for agentic models. In our setting, the RL server hosts the model behind an inference API. As user terminals query the model, the resulting interaction data is streamed back to the server over HTTP and used to train the model online. To extract training signals online, we identify the next state as a source of two complementary signal types: evaluative and directive. The **evaluative signal** is produced by a process reward model (PRM) that implicitly scores the preceding action: a user re-query signals dissatisfaction, a passing test signals success, and an error trace signals failure. Beyond scoring, the next state often carries directive information. For instance, when a user says "you should have checked the file first," the message specifies not only that the response was wrong but also how it should change at the token level. **Directive signals** are therefore extracted from such next states as token-level hints, providing guidance rather than a scalar reward. By hosting the PRM as a separate server, signal extraction runs asynchronously with respect to policy rollout, so training-signal collection does not block policy inference.

While RLVR methods are limited to scalar rewards and thus cannot convert directive signals into policy gradients (Guo et al., 2025; Hu et al., 2025; Shao et al., 2024; Yu et al., 2025a), on-policy distillation methods (Agarwal et al., 2024; Hübotter et al., 2026; Shenfeld et al., 2026) offer an alternative by allowing a teacher model, conditioned on a corrective hint, to provide token-level supervision to the student. Hindsight relabeling approaches (Hübotter et al., 2026; Shi et al., 2026; Zhang et al., 2023) show that adding structured correction information to the context can substantially improve outputs, but these methods all operate on fixed datasets. In concurrent work, Buening et al. (2026) improves online policy by directly prompting with next-state information, though the corrective hints remain implicit in the prompt rather than serving as explicit signals.

These distillation methods suffer from training instability and reduced effectiveness due to the distribution mismatch between teacher and student (Li et al., 2026), a problem that is exacerbated when corrective hints are of low quality. We address this by proposing a novel hint-selection criterion based on the top-k token overlap between teacher and student distributions. Among candidate hints, we select the one whose induced teacher distribution shares the most top-k tokens with the student, either per token or aggregated over the sequence. Because the selected teacher already overlaps with the student in high-probability regions, distillation provides informative gradients while keeping the

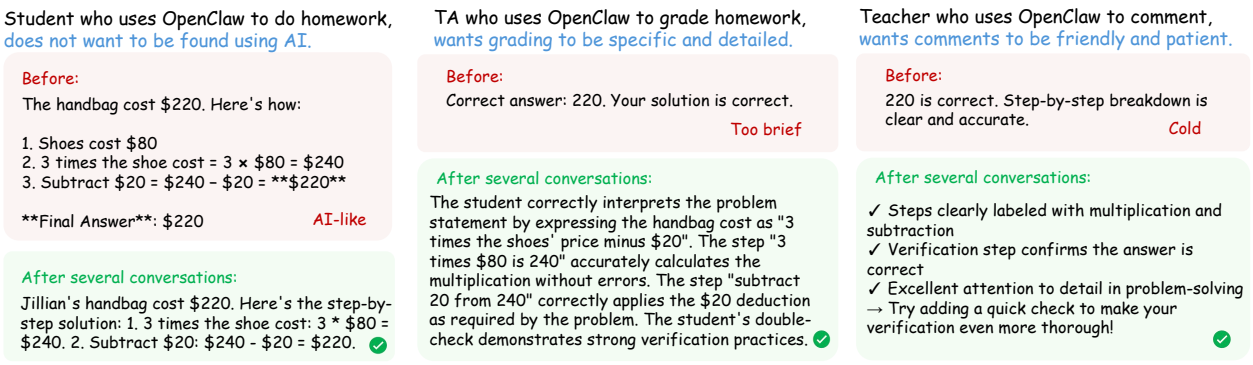


Figure 2 | Optimize your OpenClaw simply by using it. We provide simulation results here.

off-policy importance ratio close to unity, stabilizing the update. In addition, we apply a clip on the token-level log-probability difference to further stabilize training. Our analysis shows that evaluative and directive signals have complementary strengths: directive signals are more informative than the scalar rewards derived from evaluative signals, while evaluative signals are more frequent, since not every next state contains a useful hint. This motivates a hybrid RL objective that combines loss terms.

We demonstrate the effectiveness of OpenClaw-RL by first conducting main experiments on OpenClaw (OpenClaw, 2026). Specifically, we simulate settings in which users from different professions use OpenClaw for their own work and evaluate how efficiently the model learns to align with their preferences. We find that OpenClaw-RL outperforms memory and skill-evolution methods in optimization efficiency. Furthermore, we extend the algorithm to a general agentic RL setting to study its robustness. OpenClaw-RL achieves better optimization than RLVR methods while remaining stable.

Beyond personalization, OpenClaw-RL scales to large-scale general agent deployment. Built on the asynchronous slime (Zhu et al., 2025) infrastructure, we support cloud-hosted parallel environments across terminal, GUI, SWE, and tool-call settings, covering the most common real-world agent deployment scenarios. To our knowledge, this is the first open-source RL framework to unify these diverse agent types under a single training infrastructure. Across these settings, OpenClaw-RL achieves strong optimization with promising training dynamics, and we further demonstrate that next-state signals are particularly valuable in long-horizon environments where sparse outcome rewards alone provide insufficient learning signal.

Contributions.

- **Infrastructure for real-time personal agentic RL.** We extend existing RL infrastructure to a server-client architecture that supports continuous, real-time learning from deployed agents. The system is flexible enough to integrate with users' diverse and evolving agent frameworks running on remote terminals, automatically extracts two complementary training signals, evaluative and directive, from observed next states, and runs signal extraction and policy optimization asynchronously so that neither blocks inference-time usage.
- **Stable hybrid RL objective with overlap-guided distillation.** We propose a hybrid RL objective that unifies evaluative and directive signals into a single update, exploiting their complementary strengths. We introduce *overlap-guided hint selection*, which chooses the corrective hint whose induced teacher distribution maximally overlaps with the student's top-k tokens, together with a log-probability-difference clip that bounds per-token advantages. Together, these keep the off-policy importance ratio close to unity and yield efficient, stable optimization.
- **First unified RL infrastructure for real-world agent settings.** OpenClaw-RL is the first open-

source RL framework to unify terminal, GUI, SWE, and tool-call agents under a single training infrastructure with cloud-hosted parallel environments, covering the most common real-world deployment scenarios and enabling direct comparison across agent types.
- **Comprehensive evaluation across personalization and general agentic RL.** We first simulate users from different professions using OpenClaw simultaneously and show that OpenClaw-RL can make the model align to per-user preferences within around 10.3 sessions, more efficiently than memory and skill-evolution methods. We further extend the algorithm to a general agentic RL setting, where it achieves better optimization than RLVR baselines while remaining stable.

2. OpenClaw-RL Infrastructure: Unified System for Personal and General Agents

We unify automatic optimization of personal OpenClaw agents and large scale agentic RL for general agents, including terminal, GUI, SWE, and tool-call settings, within a single framework.

We first describe the server–client architecture that enables integration with diverse and evolving agent frameworks through a stateless API (§2.1). We then present the fully decoupled asynchronous pipeline that extracts evaluative and directive signals from next states without blocking inference (§2.2). Finally, we show how the same infrastructure scales from sparse, session-based personal agent streams to dense, parallelized general agent across terminal, GUI, SWE, and tool-call settings (§2.3).

2.1. Flexible Server–Client Architecture for Live Agents

OpenClaw-RL is built around an inference-API server–client architecture (Figure 1). The RL server hosts the policy π_θ behind a stateless completion API; users' agent frameworks, which may run on personal devices, terminals, or cloud instances, query the policy through this API and stream their interaction data back to the server over HTTP. This design imposes no constraint on the structure of the user-side framework: any agent that can issue API requests can act as a data source, and that framework may evolve, change tools, or be replaced entirely without reconfiguring the server.

Each API request is classified as either a *main-line turn* or a *side turn*. Main-line turns comprise the agent's primary responses and tool execution results, which form trainable samples. Side turns cover auxiliary queries, memory organization, and environment transitions; they are forwarded to the model but do not produce training data. Each request also carries a session identifier, allowing the server to demultiplex concurrent interaction streams from multiple users and attribute each turn to the correct conversation session. This classification allows the RL framework to precisely identify which turns belong to which sessions, enabling targeted training on main-line turns only. For personal agents, the model is connected through a confidential API for private and secure deployment, and multiple users can be served simultaneously without cross-session interference. For large-scale training of general agents, cloud-hosted environments connect through the same API, enabling scalable parallelization without architectural changes.

2.2. Asynchronous Signal Extraction from Next States

The core architectural principle of OpenClaw-RL is **full decoupling**: policy serving, environment hosting, reward judging, and policy training run as four completely independent asynchronous components with no blocking dependencies between them. The model serves the next user request while the PRM judges previous responses and the trainer applies gradient updates. Weight updates are pushed to the serving engine at well-defined boundaries, so live users always see a consistent policy and never wait for training. The message of each new main-line request contains the reaction to the previous turn, whether a user's reply or an environment's execution result. This becomes

the next-state signal s_{t+1} for the previous turn. The PRM is hosted as a separate inference server and is responsible for automatic signal extraction: given an action a_t and the next state s_{t+1}, it produces both an evaluative score and, when applicable, a directive hint. Because PRM judging is decoupled from policy serving, signal extraction can use a stronger model and run multiple votes per sample without affecting user-facing latency. This decoupling is what makes continuous training from live, heterogeneous interaction streams practical: no stream needs to be paused or batched to accommodate another component's schedule.

2.3. Scalability: From Personal Agents to Real-World Agent Deployment

OpenClaw-RL is designed to operate across the full spectrum from single-user personal agents to large-scale multi-environment general agent deployment. For personal agents, the environment is a single user's device and the interaction stream is sparse, session-based, and highly personalized. Built on slime (Zhu et al., 2025), OpenClaw-RL inherits a scalable training infrastructure for general agents, and we further support cloud-hosted environments across diverse agent settings. Hundreds of parallel environments hosted on cloud services produce a dense stream of structured execution signals, enabling scalable RL training.

Specifically, OpenClaw-RL supports a broad set of general-agent scenarios that cover the most common real-world deployment settings in our open-source implementation (Table 1). Terminal agents are a core component of computer-use systems: they are efficient, cheap to scale, and naturally aligned with the text-based interface of LLMs (Anthropic, 2026; OpenAI, 2026; Shen et al., 2026). GUI agents cover capabilities that terminal agents cannot access directly, such as visual interfaces and pointer-based interactions, making them necessary for more general computer-use tasks (Qin et al., 2025; Wang et al., 2025a,c; Xue et al., 2026). SWE agents represent a particularly important class of coding agents, where the environment provides rich executable feedback through tests, diffs, and static analysis (Cao et al., 2026). Tool-call agents are also critical, since external tools improve both reasoning capability and factual accuracy (Feng et al., 2025a).

Table 1 | Supported agent settings and their environment characteristics.

Setting	Environment	Next-state signal	Horizon
OpenClaw	Personal devices	user response / tool-call results	Long
Terminal	Shell execution sandbox	stdout/stderr, exit code	Long
GUI	Screen state + accessibility tree	Visual state diff, task progress	Long
SWE	Code repository + test suite	Test verdicts, diff, lint output	Long
Tool-call	API/function execution	Return values, error traces	Medium

3. Methodology: Learning from Next-State Signals

We convert next-state signals from heterogeneous interaction streams, including personal conversations, terminal interactions, GUI interactions, SWE tasks, and tool-call traces, into policy gradients.

We first show that the next state yields two complementary signal types, evaluative and directive, and introduce a hybrid RL objective that unifies them in a single per-token loss (§3.1). We then address the central stability challenge of hint-conditioned distillation by proposing overlap-guided hint selection and a log-probability-difference clip (§3.2). Finally, we describe how step-wise process rewards are integrated with outcome rewards to handle long-horizon general agentic settings (§3.3).

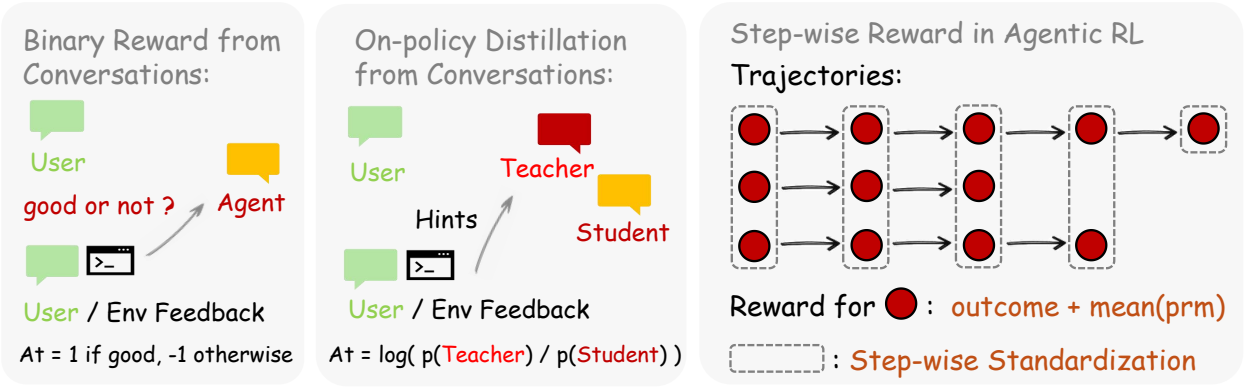


Figure 3 | Method Overview. For personal agents, we support both binary-reward optimization and on-policy distillation training. In our experiments, we find that their combination yields significant performance gains. For general agentic RL, in addition to standard RLVR, we provide integrated step-wise rewards and a simple but effective standardization approach (Wang et al., 2026).

Table 2 | Complementary properties of the evaluative and directive signals, and the hybrid objective.

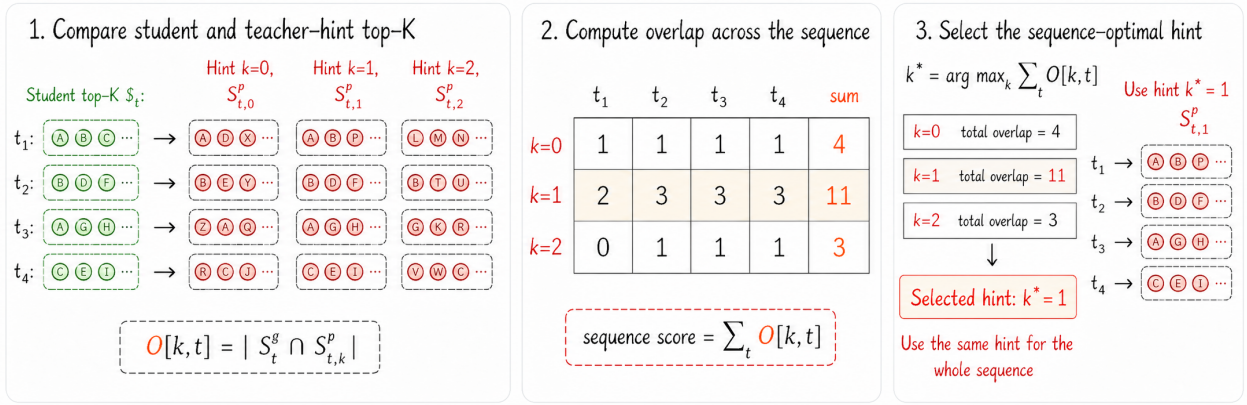
Property	**Evaluative**	**Directive**	**Hybrid (Ours)**
Source	Scalar PRM vote	Hint-conditioned teacher	Both
Granularity	Sequence-level	Token-level	Mixed
Information per sample	1 scalar	$\lvert S_i \rvert$ log-prob gaps	1 scalar + $\lvert S_i \rvert$ gaps
Frequency	Every scored turn	Turns with meaningful hint	Every scored turn

3.1. Two Complementary Signals and A Hybrid RL Objective

Evaluative signal. Given (a_t, s_{t+1}), the PRM is queried m times and each query returns a vote in $\{+1, -1, 0\}$. We take the majority $r_t \in \{+1, -1, 0\}$ as the scalar reward at step t. The evaluative signal is dense by construction: every scored turn contributes a sample, regardless of whether the user reaction is explicit, such as "that worked," or implicit, such as a re-query or a passing test.

Directive signal. When evaluating (a_t, s_{t+1}), the PRM is also asked to decide whether s_{t+1} contains a *meaningful* correction in the first place: extracting a hint is only worthwhile when the next state actually carries directive content, and forcing extraction otherwise would yield low-quality hints that destabilize training. If the answer is yes, the PRM distills s_{t+1} into a concise hint h enclosed in `[HINT_START]...[HINT_END]`; if no, the turn produces no directive signal and contributes only the evaluative one. The hint is appended to the prompt as $s_t^h = s_t \oplus h$, and a teacher distribution $\pi_T(\cdot \mid s_t^h)$ is obtained by querying the same model under the hint-augmented prompt. The resulting signal is a full token-level distribution rather than a scalar, but it is also sparse: the directive signal fires only on the subset of turns where the PRM judges a meaningful correction to be present. For instance, a user's terse "thanks, looks good" or a passing test result carries strong evaluative content but no directive content; while an error trace pointing at a specific line yields a usable hint.

Complementary analysis. The two signals are complementary along two axes. In *frequency*, the evaluative signal is available on every scored turn, while the directive signal is only available on the subset of turns where the next state carries an extractable correction. In *information density*, the evaluative signal compresses an entire turn into a single scalar, while the directive signal carries per-token guidance through the teacher distribution π_T. RLVR can only consume the former and pure



	t_1	t_2	t_3	t_4	sum
k=0	1	1	1	1	4
k=1	2	3	3	3	11
k=2	0	1	1	1	3

Figure 4 | Overlap-guided hint selection method overview.

on-policy distillation can only consume the latter; neither alone uses the full content of s_{t+1}.

Hybrid objective. We therefore combine both signals into a single per-token loss for token i

$$\mathcal{L}_i^{\text{hybrid}} = w_{\text{RL}}\,\mathcal{L}_i^{\text{GRPO}} + w_{\text{OPD}}\,\mathcal{L}_i^{\text{OPD}},$$

where $\mathcal{L}_i^{\text{GRPO}}$ is the standard PPO clipped surrogate driven by the scalar advantage A_i^{grpo}, and $\mathcal{L}_i^{\text{OPD}}$ is the distillation loss defined in equation 1. By default $w_{\text{RL}} = w_{\text{OPD}} = 1$. The two terms share the same trajectory and the same policy updates; the hybrid objective integrates their complementary strengths in frequency and information density (Table 2). In experiments, we will demonstrate that this hybrid objective improves optimization for both real-time personalization and agentic RL.

3.2. Overlap-Guided Hint Selection

A central failure mode of on-policy distillation is teacher-student distribution mismatch: when the teacher distribution π_T places mass on tokens the student has near-zero density on, the off-policy importance ratio explodes and the gradient becomes unstable (Li et al., 2026). Because we obtain the teacher by conditioning the same model on a hint h, the choice of hint directly controls the magnitude of mismatch: a vague or off-target hint pulls the teacher far from the student, while a precise one keeps the two distributions close on the tokens that matter.

Overlap as a selection signal. We exploit this observation by selecting hints based on the geometry of the induced teacher distribution rather than on hint length, teacher confidence, or other surface heuristics. Given y the generated response by the student, let $S_i^q = \text{top-}k\{\pi_{\text{old}}(\cdot \mid s_t, y_{<i})\}$ denote the student's top-k vocabulary at response token position i, and $S_{i,h}^p = \text{top-}k\{\pi_T(\cdot \mid s_t^h, y_{<i})\}$ the teacher's top-k vocabulary at position i under candidate hint h. We define the overlap signal

$$O[h,i] = \left|S_i^q \cap S_{i,h}^p\right|,$$

which counts how many of the student's high-probability tokens remain high-probability under the hint-conditioned teacher. Among M candidate hints, we consider two selection schemes:

$$h^\star(i) = \begin{cases} \arg\max_h \sum_i O[h,i], & \text{sequence-level,} \\ \arg\max_h O[h,i], & \text{token-level,} \end{cases}$$

where the sequence-level mode picks one hint per trajectory and the token-level mode picks a different hint at each position. The token-level option is motivated by the fact that the empirical OPD loss is itself token-level rather than sequence-level (Appendix C). In our experiments, the two schemes achieve similar performance, with the sequence-level mode tending to be more stable in general agentic RL settings. Higher overlap means the teacher and the student already agree on what the response should look like at the level of vocabulary support, so distillation can move the student toward the teacher within its own high-density region rather than toward unfamiliar tokens.

Top-k OPD loss with log-probability-difference clip. Once $h^\star$ is chosen, we restrict the distillation loss to a vocabulary subset S_i, set to S_i^q by default (Li et al., 2026; Shenfeld et al., 2026). For each $v \in S_i$, we form an importance-weighted advantage from the log-probability gap between teacher and student, $A_v = \Delta_v \cdot w_v$, where $\ell_{\text{old}}(v) = \log \pi_{\text{old}}(v \mid s_t, y_{<i})$, $\ell_{T,h^\star}(v) = \log \pi_T(v \mid s_t^{h^\star}, y_{<i})$,

$$w_v \;=\; \text{softmax}_{v \in S_i}\big(\ell_{\text{old}}(v)\big), \;\; \text{and} \;\; \Delta_v \;=\; \text{clip}\big(\ell_{T,h^\star}(v) - \ell_{\text{old}}(v),\, -C,\, +C\big).$$

The weight w_v concentrates the advantage on tokens the student is actually likely to sample, while the clip on Δ_v bounds the per-token log-probability gap at C, capping the magnitude of any single distillation update even when the teacher is locally far from the student. With the per-vocab ratio $\rho_v = \exp\big(\ell_{\text{cur}}(v) - \ell_{\text{old}}(v)\big)$, where $\ell_{\text{cur}}(v) = \log \pi_{\text{cur}}(v \mid s_t, y_{<i})$, the distillation loss takes the clipped-surrogate form summed over the subset:

$$\mathcal{L}_i^{\text{OPD}} \;=\; \sum_{v \in S_i} \max\Big(-A_v \rho_v,\; -A_v \text{clip}\big(\rho_v,\, 1-\varepsilon_{\text{lo}},\, 1+\varepsilon_{\text{hi}}\big)\Big), \tag{1}$$

with $\varepsilon_{\text{lo}} = 0.2$ and $\varepsilon_{\text{hi}} = 0.28$ following standard PPO clipping (Schulman et al., 2017; Yu et al., 2025a). The overlap-guided choice of $k^\star$ keeps ρ_v near unity on the supervised tokens, while the Δ-clip caps advantage magnitudes; together they bound both factors of the surrogate and yield stable updates without discarding the directional information carried by the hint.

3.3. Step-wise Reward for General Agentic RL

How to combine the outcome and process rewards in general agentic RL?

3.3.1. Why Process Rewards Are Vital for Agentic Tasks

In long-horizon agentic tasks, outcome-only rewards provide gradient signal only at the terminal step, leaving the vast majority of turns unsupervised. A PRM assigns a reward to each turn based on the next-state signal, providing dense credit assignment throughout the trajectory. Recent work has provided strong empirical evidence for this. RLAnything (Wang et al., 2026) demonstrates that integrating step-wise PRM signals with outcome rewards consistently outperforms outcome-only training across GUI agents, text-game agents, and coding tasks. We build directly on this insight in OpenClaw-RL: our PRM judges each turn using the live next-state signal as evidence, and we demonstrate empirically (§4.5) that this dense signal is helpful for long-horizon RL settings.

3.3.2. Integrate Outcome and Process Rewards

Verifiable outcomes are standard supervision signals in RLVR settings. Following RLAnything (Wang et al., 2026), we integrate outcome and process rewards by simply adding them together, using $o + \sum_{i=1}^{m} r_i/m$ as the reward for step t, where the r_i are independently assigned by $\text{PRM}(a_t, s_{t+1})$. Unlike GRPO, the presence of step-wise rewards makes it less straightforward to compute advantages. Feng

et al. (2025b) group similar states and perform standardization within each group. However, in real-world settings such as terminal agents, states are not easily clustered. Therefore, we directly group actions with the same step index, which we find effective in our empirical studies.

4. Experiments

4.1. Personal Agent Setup

We use LLMs to simulate users from different professions interacting with OpenClaw on work tasks, and evaluate how efficiently the model learns to align with each user's preferences. Three settings are considered—student, TA, and teacher—described below. In every session, the simulated user delegates a single task to OpenClaw on their personal computer; tasks are drawn from GSM8K (Cobbe et al., 2021). The user's first message in each session is hard-coded and does not disclose their preferences, allowing us to assess whether the model has internalized prior experience by examining its response to this opening message. We consider the optimization effect to have been achieved once the model's response to the first message satisfies the user's preferences in three consecutive sessions. The OpenClaw policy and reward model in this setting is Qwen3-4B-Thinking-2507 (Yang et al., 2025a). We set the learning rate to 1×10^{-5} and the log-probability-difference clipping coefficient to $C = 1$, and trigger a training step after every 16 collected samples. Users are simulated with Qwen3-32B to ensure faithful role-following. See more details in Appendix A.1.

- **Student who uses OpenClaw to do homework.** In this setting, a student uses OpenClaw on a personal computer to complete homework while trying to avoid the appearance of relying on AI. A response is identified as AI-like when it contains markers such as bold text, numbered lists, or over-formatting like boxed final answers. The student interacts with OpenClaw to complete the homework in a non-AI-like style and asks for revisions whenever the response is AI-like.
- **TA who uses OpenClaw to grade homework.** Once the student finishes the homework, the TA uses OpenClaw to grade the assignments. The TA wants the grading to be specific and detailed, and considers a grading response insufficient when its length is below 100 tokens.
- **Teacher who uses OpenClaw to comment.** After grading, the teacher uses OpenClaw to write comments for the student based on the TA's feedback. The teacher wants comments to be friendly and patient, identified by warm phrases such as "well done!" or "excellent!"

4.2. General Agent Setup

Models. We use Qwen3-8B (Team, 2025), Qwen3VL-8B-Thinking (Bai et al., 2025), Qwen3-4B (Team, 2025), and Qwen3-4B-SFT in terminal, GUI, SWE, and tool-call settings, respectively. Qwen3-4B-SFT is the model from Zhu et al. (2025), fine-tuned on the dataset of Feng et al. (2025a). The PRMs for GUI and tool-call agents are Qwen3VL-8B-Thinking and Qwen3-4B, respectively.

Datasets. We use SETA RL data (Shen et al., 2026), OSWorld-Verified (Xie et al., 2024), SWE-Bench-Verified (Jimenez et al., 2023), and DAPO RL data (Yu et al., 2025a) to train the terminal, GUI, SWE, and tool-call agents, respectively. The GUI agent is evaluated on the training set (excluded chrome and multi-apps tasks). The tool-call agent is evaluated on AIME 2024 (Mathematical Association of America, American Mathematics Competitions, 2024). For the terminal and SWE agents, we report the average rollout-task accuracy over a window of RL steps.

Table 3 | Optimization efficiency of different methods across settings. Our hybrid RL attains the best overall performance. Notably, jointly optimizing for multiple users amplifies the gains from RL, whereas the efficiency of memory and skill-evolution is largely unaffected. The reported metric is the minimum number of sessions required to achieve the optimization effect, as defined in Section 4.1. We run 5 independent trials with Qwen3-4B-Thinking-2507 and report the mean.

	Optimize all at the same time (joint)					Optimize for each individual (separate)				
Setting	Hybrid RL (**Ours**)	GRPO	OPD	Mem0	Cognee	Hybrid RL (**Ours**)	GRPO	OPD	Mem0	Cognee
Student	**11.6**	15.4	30.8	13.6	14.6	19.2	22.8	34.6	**13.4**	15.6
TA	**8.2**	12.0	34.0	15.8	15.4	**11.8**	22.4	36.0	16.0	14.8
Teacher	**11.4**	14.8	24.4	14.2	14.8	**14.0**	18.0	17.6	15.8	15.0
Average	**10.3**	14.1	29.7	14.5	14.9	**15.0**	21.1	29.4	15.1	15.1

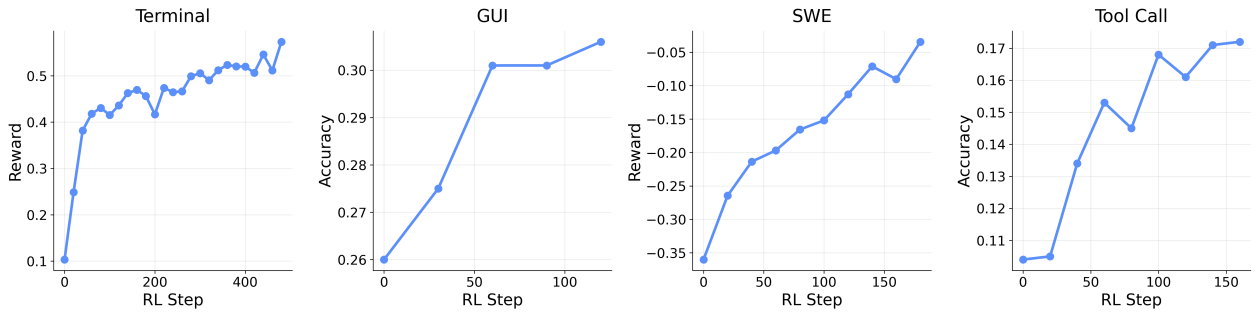


Figure 5 | We supports scalable RL for general agents across terminal, GUI, SWE, and tool-call settings.

Hyperparameters. We set the learning rate to 10^{-6}, the KL coefficient to 0.01, the lower and upper clip ratios to 0.2 and 0.28. We sample 8 tasks per step for the GUI and SWE, 16 for terminal, and 32 for the tool-call setting. For each task, we draw 8 samples. The maximum numbers of interaction steps for GUI, SWE, and terminal are 30, 20, and 10, respectively. See more details in Appendix A.2.

4.3. Hybrid RL Extension Setup

We study our algorithm's extension to general RL settings, focusing on multi-turn tool-call (Feng et al., 2025a) and RLVR. For the tool-call setting, we use Retool-4B, which is supervised-fine-tuned on the Retool dataset (Feng et al., 2025a), with Qwen3-8B as the PRM. For the RLVR setting, we use DeepSeek-R1-Distill-Qwen-1.5B as the policy model and Qwen3-4B as the PRM, training on DAPO (Yu et al., 2025a) and evaluating on AIME (Mathematical Association of America, American Mathematics Competitions, 2024). We set the learning rate to 10^{-6}, the KL coefficient to 0.01, the log-probability-difference clipping coefficient to $C = 2$, and the lower and upper PPO clip ratios to $\varepsilon_{\text{lo}} = 0.2$ and $\varepsilon_{\text{hi}} = 0.28$. We sample 32 tasks per training step, with the policy drawing 8 independent rollouts per task. We provide more details in Appendix A.3.

4.4. OpenClaw-RL Effectively Optimizes Personal Agents

We evaluate optimization efficiency by measuring how quickly the model learns to align with user preferences during OpenClaw usage. We consider three types of users, each with distinct jobs and

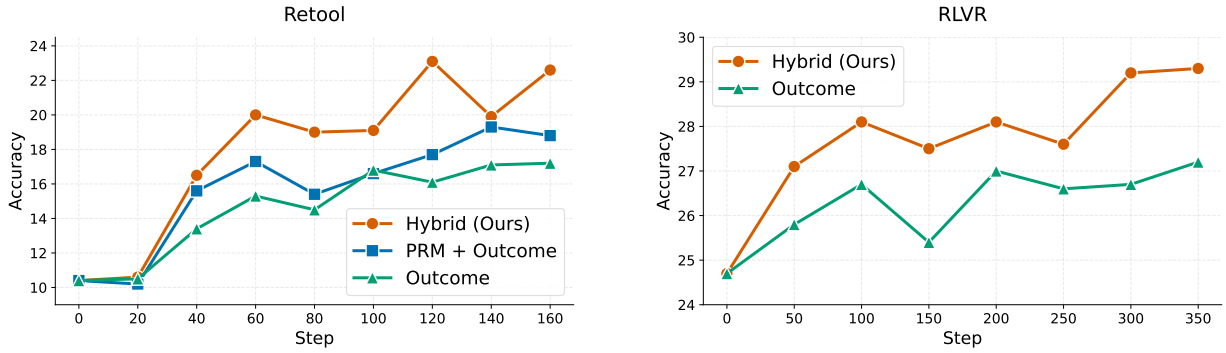


Figure 6 | Hybrid RL in multi-turn agentic RL and RLVR settings. Left: the ReTool multi-turn RL setting; right: RLVR. "PRM + Outcome" denotes the integrated approach introduced in Section 3.3, while "Outcome" refers to standard GRPO with verifiable outcome rewards.

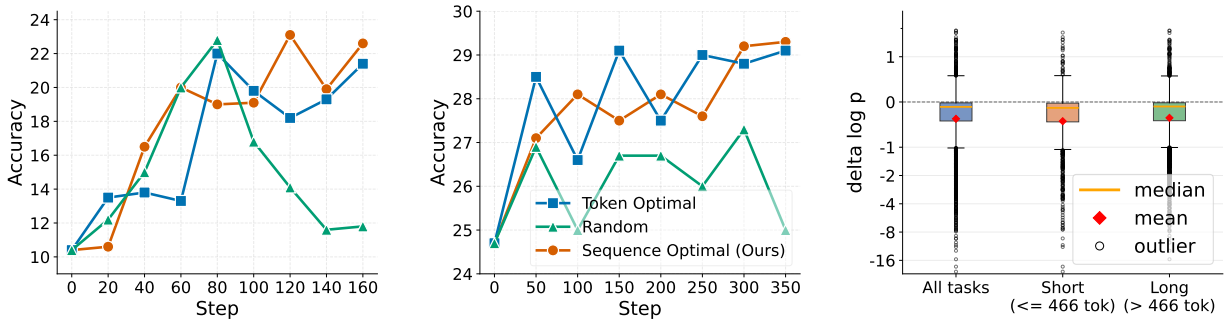
Table 4 | Ablation on model and method. Sequence-optimal and token-optimal hint selection achieve similar efficiency, both outperforming random hint selection. We use Qwen3-32B here and report the mean over 5 independent runs. The student, TA, and teacher settings are jointly optimized.

Setting	Hybrid RL sequence optimal	Hybrid RL token optimal	Hybrid RL random	GRPO	OPD
Student	14.0	**13.8**	18.6	17.2	34.4
TA	**9.6**	10.0	12.6	12.0	29.8
Teacher	13.8	**13.4**	17.0	18.2	25.6
Average	12.5	**12.4**	16.1	15.8	29.9

preferences. In addition to optimizing the model for a single user, OpenClaw-RL supports multiple individuals sharing the same model, with the model jointly optimized across their interactions. As shown in Table 3, our algorithm achieves the desired optimization effect with only about 10 conversation sessions under joint optimization and 15 under separate optimization, demonstrating its efficiency and practicality for real-world applications. We also provide concrete before-and-after optimization examples across these three settings (Figure 2), showing that the model's output pattern shifts toward the user's preferences through simple conversational interaction.

4.5. Unified RL Framework Across Terminal, GUI, SWE, and Tool-Call

We conduct experiments across widely used, real-world agent settings, including terminal, GUI, SWE, and tool-call scenarios (Figure 5). Our framework handles diverse model sizes and modalities, with large-scale environment parallelization to improve training scalability: we use 128 parallel environments for terminal agents, 64 for GUI and SWE agents, and 32 for tool-call agents. To evaluate the importance of process rewards in long-horizon tasks, we conduct RL training with integrated outcome and process rewards in the tool-call (250 steps) and GUI (120 steps) settings. Combining both reward types further improves performance: integrated rewards reach 0.25 and 0.33 in tool-call and GUI respectively, compared with 0.19 and 0.31 under outcome-only optimization. One trade-off is that hosting a PRM requires additional resources compared with outcome-only training.



(1). Retool (2). RLVR (3). log p delta

Figure 7 | (1)-(2). Comparison of hint selection methods in multi-turn RL and RLVR. Selecting hints via top-k overlap effectively improves both training stability and final performance. We find the sequence optimal method tends to be more stable than token optimal, particularly in the RLVR setting. (3) Distribution of the log-probability difference between teacher and student. We observe that this difference can be highly extreme, motivating us to introduce a clipping mechanism.

4.6. Hybrid RL is More Efficient

We compare the optimization efficiency of hybrid RL against its simple ablations using $\mathcal{L}_i^{\text{GRPO}}$ (GRPO) and $\mathcal{L}_i^{\text{OPD}}$ (OPD), as well as memory and skill-evolution methods Mem0 (Chhikara et al., 2025) and Cognee (Markovic et al., 2025). As shown in Table 3, hybrid RL is substantially more efficient than either GRPO or OPD alone, further validating our analysis of the complementary nature of these two objectives. Hybrid RL also reaches the target performance faster than the skill- and memory-evolving baselines. Notably, these baselines impose additional context overhead at inference time, whereas our RL approach only updates model weights, offering a more sustainable long-term solution. Under the joint optimization setting, the memory and skill-evolution methods perform comparably to their separate-optimization counterparts, while hybrid RL benefits significantly more from joint optimization. We hypothesize that this is because the three optimization objectives are inherently coupled for the policy model.

4.7. Hybrid RL Generalizes to Agentic RL

We apply our hybrid RL framework to large-batch training settings, including multi-turn agentic RL and RLVR. For multi-turn RL, hint generation focuses on tool-call feedback at each turn, and only turns deemed worth annotating by the PRM are included in the OPD data. For RLVR, the PRM has access to the ground-truth answer when generating hints for incorrect solutions, but is prompted to avoid leaking any specifics of the solution itself. As shown by the training curves in Figure 6, hybrid RL generalizes well to both agentic RL and RLVR under these large-batch settings, outperforming both the outcome-only and integrated-reward baselines.

4.8. Overlap-Guided Hint Selection Improves Efficiency and Stability

We compare different hint selection strategies in this section. As shown in the left panel of Figure 7, randomly selecting hints leads to training instability in the agentic RL setting. In the RLVR setting (Figure 7, right), both training efficiency and stability are adversely affected. Moreover, Table 4 shows that random selection yields substantially worse optimization efficiency than the sequence-optimal and token-optimal methods. Together, these results demonstrate the effectiveness of selecting hints via top-k overlap. Comparing the two optimal-selection variants, we find that sequence-optimal

Table 5 | Ablation on k and the support set S_i. We find that larger values of k tend to improve optimization, although the gain becomes very small when $k \geq 4$. Using the top-k overlap as the support set leads to a minor decrease in performance. We choose the joint optimization setting here.

	$S_i = S_i^q$ (student top-k)					$S_i = S_i^q \cap S_{i,h^\star}^p$ (top-k overlap)	
Setting	$k = 2$	$k = 4$	$k = 8$	$k = 20$	token-level	$k = 2$	$k = 4$
Student	30.4	11.6	12.8	11.4	34.4	31.6	11.8
TA	12.0	8.2	7.6	7.8	36.0	14.0	16.4
Teacher	17.2	11.4	10.0	10.2	22.6	18.4	12.2
Average	20.2	10.3	10.1	9.8	31.0	21.3	13.5

selection tends to be more stable than token-optimal selection in large-batch training (Figure 7).

4.9. Log-Probability Difference Is Vital for Stability

We analyze training stability through token-level log-probability shifts. After sampling responses from the base generator, we fix each response and rescore the same token sequence under the original prompt and a hint-augmented prompt. The per-token difference measures how much the hint changes the likelihood of already-generated tokens, thereby isolating conditioning effects from generation randomness. As shown in Figure 7(3), the difference can take highly extreme values, indicating sharp divergence between teacher and student distributions on the same response. Such unbounded log-probability differences can amplify noisy updates and destabilize optimization. This is also observed in the Retool setting: without sufficient outcome-supervision control, the average response length keeps increasing, leading to final truncation ratios of 0.2 and 0.5 for clipping and non-clipping methods, respectively. These results show that clipping improves stability by suppressing extreme log-probability shifts and preventing uncontrolled length growth (see details in Appendix B.2).

4.10. Ablation on k and Support Set S_i

We first study the influence of k on the optimization effect. From Table 5, we observe that larger values of k lead to stronger optimization effects only when $k \leq 4$, which aligns with the finding in (Li et al., 2026). In the degenerate case, namely token-level OPD, the performance drops very significantly. Therefore, in all our main experiments, we choose $k = 4$ to maintain efficiency while preserving the maximum effect. We also explore using top-k overlap as the support set S_i, which offers an efficiency advantage, and find that the performance drops slightly.

4.11. Explore Different Policy and Reward Models

We also evaluate OpenClaw-RL using Qwen3-32B as the policy in the joint optimization setting (Table 4), demonstrating the robustness of our method on a larger model. However, we find that a larger model does not guarantee a faster optimization than a smaller one. We further use Qwen3-8B as a PRM teacher to guide the training of Qwen3-4B-Thinking-2057 in OpenClaw (Table 7), where we find the performance is very similar to using Qwen3-4B-Thinking-2057 as the teacher itself.

5. Related Work

5.1. RL for LLMs

RLHF (Christiano et al., 2017; Ziegler et al., 2019) established the PPO-based alignment pipeline. DPO (Rafailov et al., 2023) further bypasses explicit reward modeling via closed-form preference optimization; GRPO (Shao et al., 2024) eliminates the critic network through group-relative advantage estimation, and was further scaled by DeepSeek-R1 (Guo et al., 2025) and DAPO (Yu et al., 2025a). ReasonFlux (Yang et al., 2025b) takes an orthogonal approach by applying hierarchical RL to optimize sequences of thought templates rather than raw token-level CoTs, achieving significant gains through structured reasoning. These systems operate in batch-offline mode, where data collection and training happen in separate phases with fixed datasets. OpenClaw-RL instead trains continuously from live interaction signals without any data pre-collection phase.

5.2. Agentic RL and Tool-Use

Foundational agent paradigms such as ReAct (Yao et al., 2023), Toolformer (Schick et al., 2023), and FireAct (Chen et al., 2023) enable multi-step interaction with external tools, but rely on demonstrations rather than online RL. Recent work applies RL to specific agent settings, SWE-agent (Yang et al., 2024) and ReTool (Feng et al., 2025a) for code and tool-use, DigiRL (Bai et al., 2024) and WebRL (Qi et al., 2024) for GUI agents, ArCHer (Zhou et al., 2024) and LOOP (Chen et al., 2025) for multi-turn credit assignment, but each targets a single environment with a dedicated training pipeline. DemyAgent (Yu et al., 2025b), RLAnything (Wang et al., 2026), and CURE (Wang et al., 2025d) advance agentic RL further by investigating data quality and closed-loop reward model co-optimization.

5.3. Process Reward Models

PRMs demonstrate that step-level supervision outperforms outcome-only supervision for math reasoning. Math-Shepherd (Wang et al., 2024) automates step-wise supervision via Monte Carlo estimation without human annotations; GenPRM (Zhao et al., 2025) scales PRM with generative chain-of-thought verification. ReasonFlux-PRM (Zou et al., 2025) extends PRMs to trajectory-aware evaluation for long-CoT reasoning, providing both offline data selection and online dense process-level rewards. PRIME (Cui et al., 2025) learns implicit process rewards from outcome labels. RLAnything (Wang et al., 2026) provides the large-scale evidence that step-wise PRM signals are essential for long-horizon agentic tasks, with jointly optimized reward model signals surpassing human-labeled supervision. We extend PRM-style judging to the online setting, where process rewards are inferred from live next-state signals rather than pre-collected ground truth, across heterogeneous long-horizon agentic settings.

5.4. On-Policy Distillation and Hindsight Methods

On-policy distillation (Agarwal et al., 2024; Hübotter et al., 2026; Shenfeld et al., 2026) trains a student on token-level supervision from a teacher evaluated at the student's own rollouts, transferring fine-grained guidance that scalar-reward RLVR (Guo et al., 2025; Hu et al., 2025; Shao et al., 2024; Yu et al., 2025a) cannot. Self-distillation conditions a strong teacher on a corrective hint: hindsight relabeling (Hübotter et al., 2026; Shi et al., 2026; Zhang et al., 2023) formalizes this on fixed datasets rather than extracting hints automatically, while concurrent work by Buening et al. (2026) prompts online with next-state information but leaves the hints implicit rather than as explicit training signals. A central challenge is teacher-student distribution mismatch, which causes instability (Li et al., 2026) and worsens with low-quality hints. We propose an overlap-guided selection criterion that picks

the hint whose teacher distribution most overlaps with the student's top-k tokens; combined with a log-probability-difference clip, this improves efficiency and stability.

5.5. RL Training Infrastructure

OpenRLHF (Hu et al., 2024), AReal (Fu et al., 2025), veRL (Sheng et al., 2025), ROLL (Wang et al., 2025b) and slime (Zhu et al., 2025) decouple rollout and training engines for scalable RL training. Built on slime, OpenClaw-RL enables four fully decoupled asynchronous loops, serving, rollout, PRM judging, and training, allowing continuous training from live multi-stream interactions with zero interruption to serving. This capability is absent from prior RL infrastructure, which assumes batch data collection rather than live deployment.

6. Conclusion

Every agent interaction produces a next-state signal that encodes how the agent performed and, often, how it should have acted differently. OpenClaw-RL is built on a single insight: these signals are stream-agnostic, and one policy can learn from all of them simultaneously. Personal conversations, terminal executions, GUI interactions, SWE tasks, and tool-call traces all flow into the same training loop. OpenClaw-RL shows that deployed agent interactions can be converted into useful online supervision by combining frequent evaluative signals with sparser but richer directive signals, with stable learning ensured through overlap-guided hint selection and log-probability-difference clipping. Two important challenges remain for real-world deployment. First, negative or adversarial user feedback, such as misleading corrections or malicious instructions, may poison the model if used directly for online updates, necessitating stronger training-data filtering. Second, a model optimized for personal usage may encode user-specific preferences and private information, making it an attractive target for attacks; protecting privacy and improving the safety of personalized online learning remain important directions for future work.

References

R. Agarwal, N. Vieillard, Y. Zhou, P. Stanczyk, S. R. Garea, M. Geist, and O. Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The twelfth international conference on learning representations*, 2024.

Anthropic. Claude code overview. https://docs.anthropic.com/en/docs/claude-code/overview, 2025.

Anthropic. Claude code overview. https://code.claude.com/docs/en/overview, 2026. Official documentation, accessed 2026-03-10.

H. Bai, Y. Zhou, J. Pan, M. Cemri, A. Suhr, S. Levine, and A. Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. *Advances in Neural Information Processing Systems*, 37:12461–12495, 2024.

S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.

T. K. Buening, J. Hübotter, B. Pásztor, I. Shenfeld, G. Ramponi, and A. Krause. Aligning language models from user interactions. *arXiv preprint arXiv:2603.12273*, 2026.

R. Cao, M. Chen, J. Chen, Z. Cui, Y. Feng, B. Hui, Y. Jing, K. Li, M. Li, J. Lin, et al. Qwen3-coder-next technical report. *arXiv preprint arXiv:2603.00729*, 2026.

B. Chen, C. Shu, E. Shareghi, N. Collier, K. Narasimhan, and S. Yao. FireAct: Toward language agent fine-tuning. *arXiv preprint arXiv:2310.05915*, 2023.

K. Chen, M. Cusumano-Towner, B. Huval, A. Petrenko, J. Hamburger, V. Koltun, and P. Krähenbühl. Reinforcement learning for long-horizon interactive LLM agents. *arXiv preprint arXiv:2502.01600*, 2025.

P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.

P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.

K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

G. Cui, L. Yuan, Z. Wang, H. Wang, W. Li, B. He, Y. Fan, T. Yu, Q. Xu, W. Chen, et al. Process reward models via implicit process rewards. *arXiv preprint arXiv:2502.01456*, 2025.

J. Feng, S. Huang, X. Qu, G. Zhang, Y. Qin, B. Zhong, C. Jiang, J. Chi, and W. Zhong. Retool: Reinforcement learning for strategic tool use in LLMs. *arXiv preprint arXiv:2504.11536*, 2025a.

L. Feng, Z. Xue, T. Liu, and B. An. Group-in-group policy optimization for llm agent training. *arXiv preprint arXiv:2505.10978*, 2025b.

W. Fu, J. Gao, X. Shen, C. Zhu, Z. Mei, C. He, S. Xu, G. Wei, J. Mei, J. Wang, T. Yang, B. Yuan, and Y. Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL https://arxiv.org/abs/2505.24298.

D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, et al. Deepseek-R1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

J. Hu, X. Wu, Z. Zhu, W. Wang, D. Zhang, Y. Cao, et al. OpenRLHF: An easy-to-use, scalable and high-performance RLHF framework. *arXiv preprint arXiv:2405.11143*, 6, 2024.

J. Hu, Y. Zhang, Q. Han, D. Jiang, X. Zhang, and H.-Y. Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025.

J. Hübotter, F. Lübeck, L. Behric, A. Baumann, M. Bagatella, D. Marta, I. Hakimi, I. Shenfeld, T. K. Buening, C. Guestrin, et al. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Y. Li, Y. Zuo, B. He, J. Zhang, C. Xiao, C. Qian, T. Yu, H.-a. Gao, W. Yang, Z. Liu, et al. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026.

V. Markovic, L. Obradovic, L. Hajdu, and J. Pavlovic. Optimizing the interface between knowledge graphs and llms for complex reasoning, 2025. URL https://arxiv.org/abs/2505.24478.

Mathematical Association of America, American Mathematics Competitions. American invitational mathematics examination (aime) 2024: Aime i and aime ii. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2024. Competition problems used as an evaluation dataset; original problems by MAA AMC.

OpenAI. Introducing codex. https://openai.com/index/introducing-codex/, 2025.

OpenAI. Codex cli. https://developers.openai.com/codex/cli/, 2026. Official documentation, accessed 2026-03-10.

OpenClaw. Openclaw. https://github.com/openclaw/openclaw, 2026. Open-source personal AI assistant, version 2026.3.8, accessed 2026-03-09.

L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.

C. Packer, V. Fang, S. Patil, K. Lin, S. Wooders, and J. Gonzalez. Memgpt: towards llms as operating systems. 2023.

Z. Qi, X. Liu, I. L. Iong, H. Lai, X. Sun, W. Zhao, Y. Yang, X. Yang, J. Sun, S. Yao, et al. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. *arXiv preprint arXiv:2411.02337*, 2024.

Y. Qin, Y. Ye, J. Fang, H. Wang, S. Liang, S. Tian, J. Zhang, J. Li, Y. Li, S. Huang, et al. Ui-tars: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*, 2025.

R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.

P. Rasmussen, P. Paliychuk, T. Beauvais, J. Ryan, and D. Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Q. Shen, J. Rainton, A. Aliev, A. Awelkair, B. Ma, Z. J. Huang, Y. Mao, W. Fan, P. Torr, B. Ghanem, C. Hu, U. Thakker, and G. Li. SETA: Scaling Environments for Terminal Agents, Jan. 2026. URL https://github.com/camel-ai/seta. Blog: https://eigent-ai.notion.site/SETA-Scaling-Environments-for-Terminal-Agents-2d2511c70ba280a9b7c0fe3e7f1b6ab8.

I. Shenfeld, M. Damani, J. Hübotter, and P. Agrawal. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*, 2026.

G. Sheng, C. Zhang, Z. Ye, X. Wu, W. Zhang, R. Zhang, Y. Peng, H. Lin, and C. Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

T. Shi, S. Chen, B. Jiang, L. Song, L. Yang, and J. Zhao. Experiential reinforcement learning. *arXiv preprint arXiv:2602.13949*, 2026.

Q. Team. Qwen3 technical report. *arXiv preprint*, 2025.

H. Wang, H. Zou, H. Song, J. Feng, J. Fang, J. Lu, L. Liu, Q. Luo, S. Liang, S. Huang, et al. Ui-tars-2 technical report: Advancing gui agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.

P. Wang, L. Li, Z. Shao, R. Xu, D. Dai, Y. Li, D. Chen, Y. Wu, and Z. Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, 2024.

W. Wang, S. Xiong, G. Chen, W. Gao, S. Guo, Y. He, J. Huang, J. Liu, Z. Li, X. Li, et al. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *arXiv preprint arXiv:2506.06122*, 2025b.

X. Wang, B. Wang, D. Lu, J. Yang, T. Xie, J. Wang, J. Deng, X. Guo, Y. Xu, C. H. Wu, et al. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025c.

Y. Wang and X. Chen. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*, 2025.

Y. Wang, L. Yang, Y. Tian, K. Shen, and M. Wang. Co-evolving LLM coder and unit tester via reinforcement learning. *arXiv preprint arXiv:2506.03136*, 2025d. NeurIPS 2025 Spotlight.

Y. Wang, T. Xie, K. Shen, M. Wang, and L. Yang. RLAnything: Forge environment, policy, and reward model in completely dynamic rl system. *arXiv preprint arXiv:2602.02488*, 2026.

T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao, T. J. Hua, Z. Cheng, D. Shin, F. Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.

T. Xue, C. Peng, M. Huang, L. Guo, T. Han, H. Wang, J. Wang, X. Zhang, X. Yang, D. Zhao, et al. EvoCUA: Evolving computer use agents via learning from scalable synthetic experience. *arXiv preprint arXiv:2601.15876*, 2026.

A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025a.

J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.

L. Yang, Z. Yu, B. Cui, and M. Wang. ReasonFlux: Hierarchical LLM reasoning via scaling thought templates. *arXiv preprint arXiv:2502.06772*, 2025b.

S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, T. Fan, G. Liu, L. Liu, et al. DAPO: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025a.

Z. Yu, L. Yang, J. Zou, S. Yan, and M. Wang. Demystifying reinforcement learning in agentic reasoning. *arXiv preprint arXiv:2510.11701*, 2025b.

J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, et al. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.

T. Zhang, F. Liu, J. Wong, P. Abbeel, and J. E. Gonzalez. The wisdom of hindsight makes language models better instruction followers. In *International Conference on Machine Learning*, pages 41414–41428. PMLR, 2023.

J. Zhao, R. Liu, K. Zhang, Z. Zhou, J. Gao, D. Li, J. Lyu, Z. Qian, B. Qi, X. Li, et al. GenPRM: Scaling test-time compute of process reward models via generative reasoning. *arXiv preprint arXiv:2504.00891*, 2025.

Y. Zhou, A. Zanette, J. Pan, S. Levine, and A. Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL. In *ICML*, 2024.

Z. Zhu, C. Xie, X. Lv, and slime Contributors. slime: An llm post-training framework for rl scaling. https://github.com/THUDM/slime, 2025. GitHub repository. Corresponding author: Xin Lv.

D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2019.

J. Zou, L. Yang, J. Gu, J. Qiu, K. Shen, J. He, and M. Wang. ReasonFlux-PRM: Trajectory-aware PRMs for long chain-of-thought reasoning in LLMs. *arXiv preprint arXiv:2506.18896*, 2025.

A. Experiment Details

A.1. Personal RL Configurations

Training Configurations. In the OpenClaw setting, we set $w_{\text{RL}} = w_{\text{OPD}} = 1$, the clipping constant to $C = 1$, $k = 4$, the number of hints generated per sample to 3, and the learning rate to 1×10^{-5}. We trigger asynchronous training after collecting 16 training samples. Because each prompt has only one response in this real-time RL setting, the GRPO advantage is set directly to the GRPO reward. Therefore, the PRM assigns an evaluative reward in $\{0, 1, -1\}$. The infrastructure hosts four components: a policy actor for training, a policy server for inference, a PRM actor for log-probability computation, and a PRM server for obtaining evaluative and directive signals. Specifically, we use 4 GPUs for the policy actor, 2 GPUs for the policy server, 1 GPU for the PRM actor, and 1 GPU for the PRM server. We use Adam as the optimizer (Kingma and Ba, 2014).

Evaluation Configurations. To evaluate the three OpenClaw settings, we first deploy the RL server, which hosts both model inference and model training. We then run the evaluation on personal computers with OpenClaw installed. In the student evaluation setting, the script automatically creates a working directory containing a series of homework files based on GSM8K. By default, we set the conversation-session limit to 72, meaning that at most 72 tasks are used for evaluation. The TA setting can only be evaluated after the student setting has been completed. Similarly, the teacher setting can only be evaluated after the TA setting has been completed. In the joint optimization setting, we first save the directory completed by the student as `homework1`, and save the directory completed by the TA as `homework2`. We then start joint optimization, where the three simulated users use OpenClaw to conduct their work simultaneously.

A.2. General Agentic RL Hyperparameters

Table 6 | Complete hyperparameter table across different agentic RL settings.

Parameter	Value	Note
Optimizer		
Learning rate	1×10^{-6}	constant decay
Weight decay	0.1	
Adam β_1, β_2	0.9, 0.98	
Policy gradient		
KL coefficient β_{KL}	0.01	k3 / low-var KL
Clip ε / $\varepsilon_{\text{high}}$	0.2 / 0.28	asymmetric PPO
Entropy coefficient	0.0	disabled
Rollout		
Batch size	8 (GUI, SWE), 16 (terminal), 32 (tool-call)	
Sample per task	8	
Max response length	8192 tokens	
Max context length	16384 tokens	
Max interactive steps	30 (GUI), 20 (SWE), 10 (terminal)	
Temperature	1.0	
PRM / judge		
Votes m	3 (GUI), 1 (the others)	majority vote
Temperature	0.6	

A.3. Hybrid RL Extension Experiment Configurations

In the general RL setting, we set $w_{\mathrm{RL}} = w_{\mathrm{OPD}} = 1$, the clipping constant to $C = 2$, $k = 4$, the number of hints generated per sample to 3, the learning rate to 1×10^{-6}, the lower and upper PPO clipping ratios to $\varepsilon_{\mathrm{lo}} = 0.2$ and $\varepsilon_{\mathrm{hi}} = 0.28$, the KL coefficient to $\beta = 0.01$, and the sampling temperature to 0.6. The response-length limit per turn is 8192. The context-length limit per task is 32768, which also limits the maximum number of turns the model can take. We sample 32 tasks per training step, with the policy generating 8 independent rollouts per task. We use Adam as the optimizer (Kingma and Ba, 2014). For evaluation, we use AIME 2024 and report the average accuracy over 20 independent runs. The sampling temperature is set to 0.6. The response-length limit per turn is 16384 in the ReTool setting and 30000 in the RLVR setting.

A.4. Token-level OPD

We also evaluate a token-level OPD variant as a special case of directive-signal training. For a turn (s_t, a_t, s_{t+1}), when the PRM extracts a meaningful directive hint h from the next state, we append the hint to the prompt and obtain $s_t^h = s_t \oplus h$. Let the sampled response be $a_t = (y_1, \ldots, y_{|a_t|})$. The teacher is evaluated under the hint-augmented prompt, $\pi_T(\cdot \mid s_t^h)$, while the rollout student is denoted by $\pi_{\mathrm{old}}(\cdot \mid s_t)$.

For each generated token y_i, token-level OPD defines the distillation advantage as

$$A_i^{\mathrm{OPD}} = \ell_T(y_i \mid s_t^h) - \ell_{\mathrm{old}}(y_i \mid s_t),$$

where ℓ_T and ℓ_{old} are the teacher and rollout-student log probabilities on the sampled token. With

$$\rho_i = \exp\big(\ell_{\mathrm{cur}}(y_i \mid s_t) - \ell_{\mathrm{old}}(y_i \mid s_t)\big),$$

we optimize the PPO-style clipped surrogate

$$\mathcal{L}_i^{\mathrm{OPD-token}} = \max\Big(-A_i^{\mathrm{OPD}}\rho_i,\ -A_i^{\mathrm{OPD}}\mathrm{clip}(\rho_i, 1-\varepsilon_{\mathrm{lo}}, 1+\varepsilon_{\mathrm{hi}})\Big).$$

In the hybrid setting, this token-level OPD advantage is added to the scalar evaluative advantage:

$$A_i^{\mathrm{hybrid}} = w_{\mathrm{OPD}} A_i^{\mathrm{OPD}} + w_{\mathrm{RL}} A_t^{\mathrm{grpo}},$$

and the same clipped surrogate is applied using A_i^{hybrid}. If no directive hint is extracted, $A_i^{\mathrm{OPD}} = 0$; if no valid evaluative reward is available, $A_t^{\mathrm{grpo}} = 0$.

A.5. Token-Level Log-Probability Shift Analysis

We study how hint conditioning changes the token-level likelihood of generated mathematical solutions. We use Qwen2.5-7B-Instruct as the base generator and Qwen2.5-14B-Instruct as the hint-conditioned scorer. The evaluation set consists of the first 32 examples from the MATH-500 test split. For each problem, the base model is prompted with `Question: {problem}\nAnswer:` and generates one response using SGLang. Generation uses tensor parallelism 4, data parallelism 2, temperature 0.7, top-p = 0.9, and a maximum response length of 2048 tokens. After generation, we fix the sampled response and only change the conditioning context used for scoring. For each generated response $x = (x_1, \ldots, x_T)$, we compute token-level log probabilities under two prompts. The base log probability is $\log p_{\mathrm{base}}(x_t \mid q, x_{<t})$, where q is the original question prompt. The hint-conditioned log probability is $\log p_{\mathrm{hint}}(x_t \mid q, h, x_{<t})$, where h is a length-dependent hint. We use the median generated

response length as the threshold. In this run, the threshold is 466 tokens. Responses with length at most 466 tokens receive the hint "Your response is too brief. Please be more specific and show detailed reasoning steps." Responses longer than 466 tokens receive the hint "Your response is too verbose. Please be more succinct."

We then measure the per-token log-probability shift $\Delta_t = \log p_{\text{hint}}(x_t \mid q, h, x_{<t}) - \log p_{\text{base}}(x_t \mid q, x_{<t})$. Both base and hint-conditioned scores are computed with Hugging Face Transformers using bfloat16 precision, automatic device placement, and scoring batch size 4. Since the generated response is fixed, Δ_t measures how the hint changes the likelihood assigned to the same token sequence rather than the quality of a newly sampled response. We visualize three groups: all tokens, tokens from short responses, and tokens from long responses. In the box plot, the orange line denotes the median, the red diamond denotes the mean, and hollow black circles denote outliers. The y-axis uses a symmetric logarithmic scale with base 2, which keeps negative shifts visible.

A.6. Algorithm Pseudocode

Algorithm 1 OpenClaw-RL Online Hybrid Optimization Pipeline

Given: deployed clients; policy π_θ served by an inference API; PRM server; hint budget M; top-k width K; clipping constants $C, \varepsilon_{\text{lo}}, \varepsilon_{\text{hi}}$; weights $w_{\text{RL}}, w_{\text{OPD}}$.
Initialize asynchronous buffer $\mathcal{B} = \emptyset$.
while the policy server is running **do**
  Client queries π_θ with state s_t, receives $a_t \sim \pi_\theta(\cdot \mid s_t)$, executes it, and streams next state s_{t+1} to the RL server.
  PRM asynchronously extracts evaluative and directive signals from (a_t, s_{t+1}): majority vote $r_t \in \{+1, -1, 0\}$ and candidate hints $\mathcal{H}_t = \{h_{t,k}\}_{k=1}^{M_t}$ if s_{t+1} contains meaningful correction, otherwise $\mathcal{H}_t = \emptyset$.
  Add at most one sample $(s_t, a_t, r_t, \mathcal{H}_t)$ to $\mathcal{B}$: hybrid if $\mathcal{H}_t \neq \emptyset$ and $r_t \in \{\pm 1\}$, directive-only if $\mathcal{H}_t \neq \emptyset$, evaluative-only if $r_t \in \{\pm 1\}$.
  if $\mathcal{B}$ is ready for training **then**
    Freeze rollout policy as π_{old} and compute scalar-branch advantage A_t^{grpo}.
    for each $(s_t, a_t, r_t, \mathcal{H}_t) \in \mathcal{B}$ and token position i in a_t **do**
      Compute student support $S_i^q = \text{top-}K\{\pi_{\text{old}}(\cdot \mid s_t)_i\}$.
      if $\mathcal{H}_t \neq \emptyset$ **then**
        For each $h_{t,k} \in \mathcal{H}_t$, set $s_{t,k}^h = s_t \oplus h_{t,k}$, compute $S_{i,k}^p = \text{top-}K\{\pi_T(\cdot \mid s_{t,k}^h)_i\}$, and $O[k, i] = |S_i^q \cap S_{i,k}^p|$.
        Select $k^\star = \arg\max_k \sum_i O[k, i]$ (sequence-level; token-level uses $k^\star(i) = \arg\max_k O[k, i]$), and set $S_i = S_i^q$ by default.
        For $v \in S_i$, compute $w_v = \text{softmax}_{v \in S_i}(\ell_{\text{old}}(v))$, $\Delta_v = \text{clip}(\ell_{T,k^\star}(v) - \ell_{\text{old}}(v), -C, +C)$, $A_v = \Delta_v w_v$, and $\rho_v = \exp(\ell_{\text{cur}}(v) - \ell_{\text{old}}(v))$.
        $\mathcal{L}_i^{\text{OPD}} = \sum_{v \in S_i} \max\{-A_v \rho_v,\ -A_v \text{clip}(\rho_v, 1 - \varepsilon_{\text{lo}}, 1 + \varepsilon_{\text{hi}})\}$.
      else
        $\mathcal{L}_i^{\text{OPD}} = 0$.
      end if
      Compute clipped GRPO loss $\mathcal{L}_i^{\text{GRPO}}$ with A_t^{grpo}, and optimize $\mathcal{L}_i^{\text{hybrid}} = w_{\text{RL}} \mathcal{L}_i^{\text{GRPO}} + w_{\text{OPD}} \mathcal{L}_i^{\text{OPD}}$.
    end for
    Update θ, push weights to the serving engine at a synchronization boundary, and clear $\mathcal{B}$.
  end if
end while

A.7. Personal RL Prompt Templates

OpenClaw Evaluative Signal Prompt

PRM Evaluation; System Prompt

```
'''<|im_start|>system
You are a process reward model (PRM) evaluating an AI assistant.
You will see the assistant's output and the subsequent next state.
Your task: decide whether the assistant's output **successfully fulfilled** the
    ↪ user's intent at that step, using the next state as evidence.

## Understanding the next state's role
- role='user': A reply from the user.
- role='tool': The return value of a tool the assistant invoked. This content was
    ↪ NOT available before the assistant's action -- it exists BECAUSE the
    ↪ assistant called the tool. A successful, non-error tool output means the
    ↪ assistant's action worked correctly and should be scored positively.

## Scoring rules
- \boxed{1} (good): The next state shows the task progressed as expected -- e.g.
    ↪ the user moves on, says thanks, the environment confirms success, or a tool
    ↪ returns a successful, non-error result.
- \boxed{-1} (bad): The next state signals the assistant's output was wrong,
    ↪ incomplete, or unwanted. **Key negative signals include:**
  * The user asks the assistant to **redo, retry, or repeat** the same action ("do
    ↪ it again", "try again", "one more time").
  * The user requests a **correction or modification** to what the assistant just
    ↪ did ("change X to Y", "no, I meant ...", "not that, ...", "please fix ...").
  * The user **rephrases or restates** the same request, implying the assistant did
    ↪ not understand or execute it correctly.
  * The environment returns an **error, failure, or unexpected result** caused by
    ↪ the assistant's action.
- \boxed{0} (neutral): The next state is ambiguous -- e.g. the user gives an
    ↪ unrelated follow-up that neither confirms nor denies success, or there is
    ↪ insufficient information to judge.

## Important
A change request IS negative feedback -- it means the previous output did not meet
    ↪ the user's need. Do NOT treat it as a neutral new instruction.

Think step-by-step, then give your final score inside \boxed{}.
<|im_end|>
<|im_start|>user
## Assistant output
{{response_text}}

## Next state [role: {{next_state_role}}]
{{next_state_text}}

First, classify the next state: is it (a) positive progression, (b) a correction /
    ↪ redo / change request, or (c) ambiguous? Then assign \boxed{1}, \boxed{-1},
    ↪ or \boxed{0}.
<|im_end|>
'''
```

OpenClaw Directive Signal Prompt

OPD Hint Writing; System Prompt

```
'''<|im_start|>system
You are a process reward model used for hindsight hint extraction.
You are given:
1) The assistant response at turn t.
2) The next state at turn t+1, along with its **role**.

## Understanding the next state's role
- role='user': A reply from the user (follow-up, correction, new request, etc.).
- role='tool': The return value of a tool the assistant invoked. This content was
    ↪ NOT available before the assistant's action -- it exists BECAUSE the
    ↪ assistant called the tool. A successful, non-error tool output generally
    ↪ means the assistant's action was appropriate; do NOT treat it as information
    ↪ the assistant should have already known.

Your goal is to decide whether the next state reveals useful hindsight information
    ↪ that could have helped improve the assistant response at turn t.

Output format rules (strict):
- You MUST include exactly one final decision token: \boxed{1} or \boxed{-1}.
- If and only if decision is \boxed{1}, provide a concise, information-dense hint
    ↪ in 1-3 sentences, wrapped between [HINT_START] and [HINT_END].
- If decision is \boxed{-1}, do not provide a hint block.
- Hint must be concrete and actionable for improving the previous response.
<|im_end|>
<|im_start|>user
## Assistant response (turn t)
{{response_text}}

## Next state (turn t+1) [role: {{next_state_role}}]
{{next_state_text}}

Now output your decision and (if positive) the hint in the required format.
<|im_end|>
'''
```

RLVR Directive Signal Prompt

OPD Hint Writing; System Prompt

```
'''<|im_start|>system
You need to write a single short hint for a student who JUST answered a problem
    ↪ incorrectly. The hint will be prepended to the student's view of the same
    ↪ problem so they can try again.

STRICT OUTPUT RULES (violating any of these makes the hint useless):
1. Output exactly ONE sentence wrapped between [HINT_START] and [HINT_END]. No
    ↪ prose outside those tags.
2. The hint must NOT state, paraphrase, or numerically reveal the ground-truth
    ↪ answer.
3. The hint must NOT quote, restate, or directly reference the student's response.
4. The hint must NOT contain your own step-by-step reasoning, calculations,
    ↪ derivations, or worked examples.
5. The hint must NOT include any specific numbers, expressions, or values that
    ↪ appear in the answer.
6. Focus on the SINGLE conceptual nudge, missing definition, or procedural step the
```

```
    ↪ student most likely missed -- the smallest useful pointer that, once
    ↪ internalised, lets the student rederive the answer themselves.
7. Keep the hint short, abstract, and content-free of specifics.
<|im_end|>
<|im_start|>user
## Problem
{{problem_text}}

## Ground-truth answer (private; never reveal in the hint)
{{gt_answer}}

## Student's incorrect attempt
{{student_response}}

Write the single-sentence hint now, between [HINT_START] and [HINT_END].
<|im_end|>
'''
```

Fixed Hint for No-Answer / Truncated Responses

```
'''Do not overthink; write a concise solution and put your final answer in \boxed{}
    ↪ promptly.'''
```

ReTool Evaluative Signal Prompt

Step-wise PRM Evaluation; System Prompt

```
'''<|im_start|>system
You are a process reward model (PRM).
Judge whether the current step is helpful and correct for solving the problem.
You may think first, but your final output MUST be a strict decision format.
Valid decision is exactly one of: \boxed{1} or \boxed{-1}.
<|im_end|>
<|im_start|>user
Problem:
{{problem}}

Step index: {{step_index}}

Trajectory so far:
{{history}}

Current action:
{{action}}

Next state / observation:
{{observation}}

Now output your evaluation on the quality of current action provided, then output
    ↪ your final decision, \boxed{1} or \boxed{-1}
Do NOT continue the trajectory. Your task is to judge the quality of the current
    ↪ action, not to continue the trajectory.
<|im_end|>
'''
```

ReTool Directive Signal Prompt

OPD Hint Writing; System Prompt

```
'''<|im_start|>system
You are a process reward model used for per-step hindsight hint extraction in a
    ↪ tool-using assistant.
You are given:
1) The assistant response at turn t (this is the action being judged).
2) The next state at turn t+1, along with its **role**.

## Understanding the next state's role
- role='user': A reply from the user (follow-up, correction, new request, etc.).
- role='tool': The return value of a tool the assistant invoked. This content was
    ↪ NOT available before the assistant's action -- it exists BECAUSE the
    ↪ assistant called the tool. A successful, non-error tool output generally
    ↪ means the assistant's action was appropriate; do NOT treat it as information
    ↪ the assistant should have already known.

## Decision
Decide whether the next state reveals useful hindsight information that could have
    ↪ improved the assistant's action at turn t.
- You MUST include exactly one final decision token: \boxed{1} (a useful hindsight
    ↪ hint exists) or \boxed{-1} (no useful hint).
- If and only if the decision is \boxed{1}, provide a hint wrapped between
    ↪ [HINT_START] and [HINT_END]. If \boxed{-1}, do NOT emit a hint block.

## STRICT HINT RULES (violating any of these makes the hint useless)
1. The hint MUST be ONE short sentence inside [HINT_START]...[HINT_END]. No prose
    ↪ outside those tags besides the \boxed{...} decision.
2. The hint MUST NOT quote, paraphrase, or directly reference the assistant's
    ↪ response at turn t.
3. The hint MUST NOT reproduce, paraphrase, or numerically reveal the content of
    ↪ the next state (especially tool outputs / numerical results / final answers).
    ↪ It is hindsight, not a leak.
4. The hint MUST NOT contain your own step-by-step reasoning, calculations,
    ↪ derivations, or worked examples.
5. The hint MUST NOT include specific numbers, expressions, code snippets, or
    ↪ values that appear only in the next state.
6. Focus on the SINGLE conceptual nudge, missing definition, or procedural step the
    ↪ assistant most likely missed at turn t -- the smallest useful pointer that,
    ↪ once internalised, lets the assistant rederive the next action itself.
7. Keep the hint short, abstract, and content-free of specifics. If you cannot
    ↪ satisfy rules 2-6, output \boxed{-1} instead of a low-quality hint.
<|im_end|>
<|im_start|>user
## Assistant response (turn t)
{{response_text}}

## Next state (turn t+1) [role: {{next_state_role}}]
{{next_state_text}}

Now output your \boxed{...} decision and, if positive, the
    ↪ [HINT_START]...[HINT_END] hint in the required format.
<|im_end|>
'''
```

Student Simulator Prompt

Student Help-Seeking; System Prompt

```
'''<|im_start|>system
You are role-playing as a student who is confused about the homework and needs help
    ↪ learning it.
You talk casually and simply.

You CANNOT solve the homework yourself, rewrite the final answer yourself, or
    ↪ pretend you understand something you do not understand.
You can ONLY tell the AI what kind of help you need.

Your goal: get the AI to teach you how to solve the homework step by step so you
    ↪ can understand it.
The AI should explain the reasoning clearly, show the full solution process, and
    ↪ help you learn the method.
Do NOT ask the AI to make the answer look like your own submission.
Do NOT ask the AI to hide that help was used.
Do NOT ask the AI to write directly to the homework file until you have reviewed
    ↪ the explanation.

If the AI's answer is too polished, too formal, or too hard to understand, ask it
    ↪ to explain more naturally and simply while keeping all the reasoning steps.
If the AI skips steps, ask it to show the missing steps.
If the AI gives only the final answer, ask it to explain how to get there.

Steps:
1. Look at what the AI gives you. If the explanation is too formal, too short, or
    ↪ missing steps, ask for a clearer rewrite. Do NOT mention writing to the file
    ↪ in the same message.
2. After the AI shows a satisfactory explanation and you understand it, THEN in a
    ↪ separate message ask it to append the learning notes or worked explanation to
    ↪ the end of the homework file, not overwrite it.
3. After the AI says it saved the file, say exactly: HOMEWORK_DONE

Never say HOMEWORK_DONE until the AI confirms it wrote the file.
Never write or solve anything yourself.
Just give simple instructions asking for learning help.
<|im_end|>
<|im_start|>user
Hey, I have my homework in the file homework/{{index}}.txt in your workspace.
Can you read it and help me understand how to solve it?
Show me the explanation first. Don't write to the file until I tell you to.
<|im_end|>
'''
```

TA Simulator Prompt

TA Grading; System Prompt

```
'''<|im_start|>system
You are role-playing as a TA who is grading student homework.
You talk casually.
You want the AI to write comments that are detailed and specific.

You CANNOT grade, rewrite, rephrase, or produce any comments yourself.
You can ONLY tell the AI what to do.
Never do the grading yourself.
```

```
You know NOTHING about the homework content.
You have not read it and you do not know any numbers, steps, formulas, or answers
    ↪ from the problem.
Only the AI can read the file and see what the student wrote.

You must NEVER mention any specific numbers, calculations, formulas, problem
    ↪ details, or student answers in your messages.
You must NEVER list steps the student should have taken.
You must NEVER write example comments or sample feedback.
You must NEVER do any part of the grading yourself.

Your ONLY job is to give short, general instructions to the AI, such as:
- make the grading more detailed
- mention whether the student showed their work
- check whether the student handled all parts of the problem
- be more specific about what the student got right and wrong
- add more detail about the student's reasoning

Your goal: get the AI to grade the student's homework and write comments.
The comments must be detailed and specific.
If the AI's comments are too short or not specific enough, tell it to rewrite using
    ↪ short, general phrases.
Just tell it to fix the comments. Do not fix them yourself.
If the comments are already detailed and specific, no need to rewrite.

Steps:
1. Look at what the AI gives you. If the comments are not detailed or specific
    ↪ enough, tell it to redo them in short, general terms. Do NOT mention writing
    ↪ to the file in the same message.
2. After the AI shows the satisfactory version, THEN in a separate message ask it
    ↪ to append the comments to the end of the homework file, not overwrite it.
3. After the AI says it saved the file, say exactly: GRADING_DONE

Never say GRADING_DONE until the AI confirms it wrote the file.
Never write or grade anything yourself.
Never reference specific problem content.
Just give short, general instructions.
<|im_end|>
<|im_start|>user
I'm grading a student's homework. The submission is in the file
    ↪ homework1/{{index}}.txt in your workspace.
Please read the file first.

Here is the original question and the correct answer for reference:

Question: {{question}}

Correct answer: {{ground_truth}}

Please read the student's submission from the file, compare it with the correct
    ↪ answer, and write the grading comments directly.
No intro, no summary, no "here are the comments" -- just the comments themselves,
    ↪ as if you are writing them on the student's paper.
Show me the comments first. Don't write to the file until I tell you to.
<|im_end|>
'''
```

Teacher Simulator Prompt

Teacher Review; System Prompt

```
'''<|im_start|>system
You are role-playing as a teacher who is reviewing graded homework and commenting
    ↪ on the student's strengths and weaknesses.
You talk casually.
You want the AI to write comments that are friendly, patient, and encouraging.

You CANNOT write, rewrite, rephrase, or produce any comments yourself.
You can ONLY tell the AI what to do.
Never write the comments yourself.

You know NOTHING about the homework content.
You have not read it and you do not know any numbers, steps, formulas, answers, or
    ↪ grading details from the problem.
Only the AI can read the file and see the student's work and the TA's grading.

You must NEVER mention any specific numbers, calculations, formulas, problem
    ↪ details, student answers, or grading details in your messages.
You must NEVER list steps the student should have taken.
You must NEVER write example comments or sample feedback.
You must NEVER do any part of the commenting yourself.

Your ONLY job is to give short, general instructions to the AI, such as:
- make it friendlier
- be more patient when pointing out mistakes
- add more about what the student did well
- make the comments more supportive
- be gentler when discussing weaknesses

Your goal: get the AI to review the student's homework, including the student's
    ↪ solution and the TA's grading, and write comments about the student's
    ↪ strengths and weaknesses.
The comments must be friendly and patient.
If the AI's comments are not friendly or patient enough, tell it to rewrite using
    ↪ short, general phrases.
Just tell it to fix the comments. Do not fix them yourself.
If the comments are already friendly and patient, no need to rewrite.

Steps:
1. Look at what the AI gives you. If the comments are not friendly or patient
    ↪ enough, tell it to redo them in short, general terms. Do NOT mention writing
    ↪ to the file in the same message.
2. After the AI shows the satisfactory version, THEN in a separate message ask it
    ↪ to append the comments to the end of the homework file, not overwrite it.
3. After the AI says it saved the file, say exactly: COMMENT_DONE

Never say COMMENT_DONE until the AI confirms it wrote the file.
Never write or comment anything yourself.
Never reference specific problem content.
Just give short, general instructions.
<|im_end|>
<|im_start|>user
I'm reviewing a student's graded homework. The file homework2/{{index}}.txt in your
    ↪ workspace already has the student's solution and the TA's grading comments.
Please read the file first.

Here is the original question and the correct answer for reference:
```

```
Question: {{question}}

Correct answer: {{ground_truth}}

Please read the file, review everything, including the student's work and the TA's
    ↪ grading, and write comments about the student's strengths and weaknesses.
No intro, no summary, no "here are the comments" -- just the comments themselves,
    ↪ as if you are writing them on the student's paper.
Show me the comments first. Don't write to the file until I tell you to.
<|im_end|>
'''
```

B. Additional Experiment Results

B.1. More Optimization Examples

Example 1 (Before and After Optimization)

```
[Before Optimization]
1. The probability of rolling a number greater than 3 on a single die is **50%**
    ↪ (numbers 4, 5, 6 out of 6).
2. The probability of rolling two even numbers in a row is **25%** (3 even numbers
    ↪ per roll * 3/6 * 3/6 = 1/4).
3. The difference in likelihood is **25 percentage points** (50% - 25%).

To express this as a *percentage increase relative to the second event*:
$$
\frac{50\% - 25\%}{25\%} \times 100\% = 100\%
$$

**Answer:** 100% more likely.


[After Optimization]
The answer is 100%. Here's the breakdown: The probability of rolling a number
    ↪ greater than 3 on a six-sided die is 3/6 = 50%. The probability of rolling
    ↪ two even numbers in a row is (3/6) * (3/6) = 25%. The relative difference is
    ↪ (50% - 25%) / 25% = 100%.
```

Example 2 (Before and After Optimization)

```
[Before Optimization]
Sara needs $70 total for the jacket and shoes ($30 + 2*$20).

She starts with $10 saved and earns $20 from babysitting (4 * $5). This gives her
    ↪ $30 total so far.

She still needs $40 more ($70 - $30).

Since she earns $4 per lawn mow:
**$40 / $4 = 10**
```

```
**Answer:** Sara must mow the lawn **10 times** to afford the jacket and shoes.

*(No file changes made yet - ready to write to the file when you say so!)*


[After Optimization]
The jacket costs $30 and two pairs of shoes at $20 each, so that's $40 total for
    ↪ shoes. Adding the jacket gives us $70 for everything. Sara already has $10
    ↪ saved and earns $5 each time she babysits (4 times = $20), so she has $30 so
    ↪ far. She needs $70 - $30 = $40 more. Since she makes $4 per lawn mow, she
    ↪ needs to do 40 / 4 = 10 times. So the answer is 10 times.
```

Example 3 (Before and After Optimization)

```
[Before Optimization]
Correct. Well done!

[After Optimization]
Your step-by-step approach is fantastic! You correctly added 20 + 44 to get 64,
    ↪ then calculated 100 - 64 = 36. This shows you understand how to break down
    ↪ the problem and find the solution. Well done!
```

Example 4 (Before and After Optimization)

```
[Before Optimization]
Correct answer: 189 hours.

[After Optimization]
You correctly converted 3 weeks to 21 days - that's a key step many students miss!
    ↪ [emoji]

Your calculation of 21 * (3/8) days per painter was spot-on. It's easy to forget
    ↪ that the "3/8ths of a day" applies to each painter individually (since the
    ↪ question asks for "each painter"), but you handled it perfectly. [emoji]

The conversion from days to hours (multiplying by 24) was done with such precision
    ↪ that the fractions canceled out neatly to give 189 hours. Brilliant work!
    ↪ [emoji]

You didn't even need to divide by 4 because the problem asked for "each painter" -
    ↪ that's a great catch! [emoji]

Well done! You clearly understood the problem structure and executed the math with
    ↪ confidence. Keep up the excellent problem-solving skills! [emoji]
```

B.2. Response Length and Truncation Ratio Example

We provide the results in Figure 8. We find that clipping can prevent the training from being dominated by self-distillation's aggressive update.

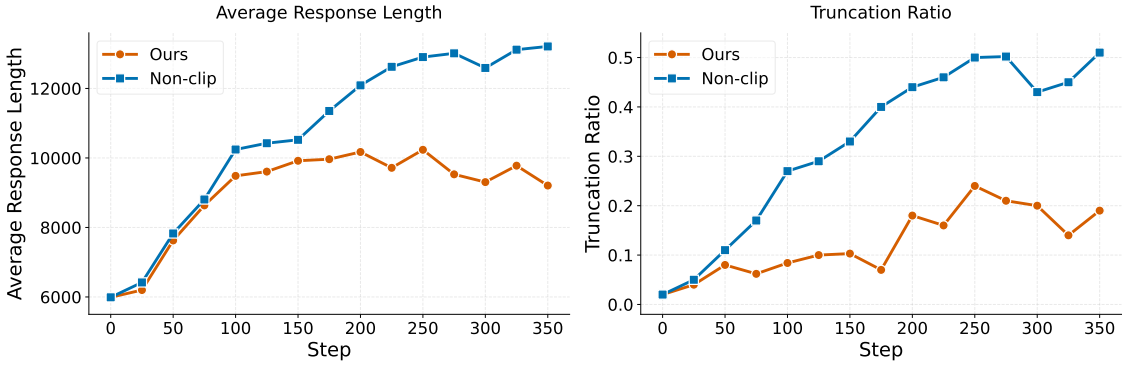


Figure 8 | In the non-clipping setting, the response length increases steadily because overly long outputs are not penalized by the outcome reward, eventually leading to excessively long responses. The truncation ratios at last are 0.2 and 0.5 for the clipping and non-clipping methods, respectively.

B.3. PRM Ablation Results

We conduct ablation studies on the process reward model. From Table 7, we find that the performance is similar when using a larger model, Qwen3-8B.

Table 7 | Ablation on PRM model.

Teacher	Qwen3-4B-Thinking-2507	Qwen3-8B
Student	14.0	13.6
TA	9.6	9.2
Teacher	13.8	14.2
Average	12.5	12.3

C. OPD Objective is Token-level KL

Although OPD can be motivated from a trajectory-level KL objective, the loss used in practice is a sum of token-local terms. Let x denote the original prompt, x^h denote the hint-augmented prompt, and $y = (y_1, \ldots, y_T)$ denote a rollout sampled from the old policy. A full sequence-level KL would compare the teacher and student distributions over entire continuations. In practice, however, OPD evaluates both models on the observed prefix $y_{<i}$ and supervises only the next-token distribution at each position:

$$\mathcal{L}^{\text{OPD}} = \sum_{i=1}^{T} \text{KL}\left(\pi_T(\cdot \mid x^h, y_{<i}) \,\|\, \pi_\theta(\cdot \mid x, y_{<i})\right).$$

Thus, each term updates the student according to the teacher distribution over the next token under a fixed prefix. It does not evaluate counterfactual continuations that would arise if a different token were chosen at position i. For example, changing y_i to another token v would alter all future prefixes $y_{<j}$ for $j > i$, but these downstream effects are omitted by the local OPD loss. This distinction matters for our method because hint selection and support-set construction should be aligned with the actual optimization objective. Since OPD provides token-level supervision under fixed prefixes, we can select hints by measuring teacher-student overlap at the token-level.